

Service-oriented Software Engineering (SoSE)

Spring Framework and Dependency Injection

Prof. Marco Autili

University of L'Aquila

marco.autili@univaq.it

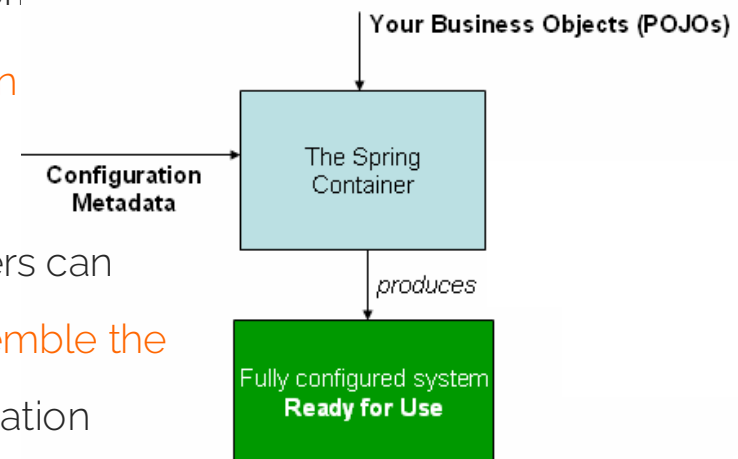
<http://people.disim.univaq.it/marco.autili/>

Spring Framework

- The Spring Framework provides a comprehensive programming and configuration model for modern **Java-based enterprise applications** - on any kind of deployment platform
 - Developers can focus on application-level business logic, without unnecessary ties to specific deployment environments
- Spring comprises a number of core technologies:
 - the main one is the **Spring Framework's Inversion of Control (IoC) container**, often referred to as **Dependency Injection (DI)** -- Precisely, **Dependency Injection** is the design pattern used by Spring to realize a specialized form of **Inversion of Control**
- The IoC process is about **objects** and **their dependencies**, i.e., **objects that define the other objects they work with** only through:
 1. constructor arguments
 2. arguments to a factory method,
 3. properties that are set on the object instance after it is constructed or returned from a factory method

IoC aka DI

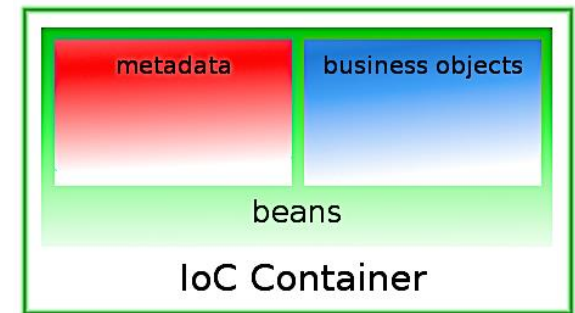
- In Spring, the **objects** that form the backbone of your application and that are managed by the Spring IoC container are called **beans**
 - beans are objects that are **instantiated**, **assembled**, and **managed** by a Spring IoC container
 - beans and the **dependencies** among them are **reflected** in the configuration metadata used by a container
 - The container then **injects those dependencies when it creates the bean**
- This process is fundamentally **the inverse** (hence the name, Inversion of Control) of the bean itself, controlling the instantiation or location of its dependencies by using direct construction of classes or a mechanism such as the Service Locator pattern
- Basically, the Spring IoC container **consumes configuration metadata**
- This configuration metadata represents the way developers can tell the Spring container **to instantiate**, **configure**, and **assemble the objects** (i.e., Plain Old Java Objects - **POJOs**) in your application



Config meta-data specification alternatives

- Spring offers two alternatives to configure **beans**:

1. XML configuration file
2. Java-based configuration



- If you are comfortable with one of the two configuration alternatives, then you not required to learn how to proceed with the other one:
 - you achieve the same result using either of the configurations available
- Java-based configuration option enables you to write most of your Spring configuration without XML but with the help of few **Java-based annotations**

For our example, we will use Java-based configuration

Configuration equivalence example

```
import org.springframework.context.annotation.*;

@Configuration
public class HelloWorldConfig {
    @Bean
    public HelloWorld helloWorld(){
        return new HelloWorld();
    }
}
```

A configuration class can declare more than one @Bean

The Java code above is equivalent to the following XML configuration:

- the **method name** annotated with **@Bean** works as **bean ID**, and it **creates** and **returns** the actual bean

```
<beans>
  <bean id = "helloWorld" class = "com.tutorialspoint.HelloWorld" />
</beans>
```

- A **class** annotated with **@Configuration** can be used by the Spring Inversion of Control (IoC) container as a source of bean definitions
- The **object returned** by a **method** annotated with **@Bean** is **registered** as a bean in the **Spring** application context

The Spring ApplicationContext (summing up)

- Spring `ApplicationContext` is where Spring holds `instances of objects` (called `beans`) that it has identified to be managed and distributed automatically
- Using the `Inversion of Control principle`, Spring collects bean instances from our application and uses them at the appropriate time
- The ability to use annotations like `@Autowired` to inject Spring-managed beans into our application is a driving force for creating powerful and scalable code in Spring

@Component VS @Bean

@Component or @Bean, the result is always a Bean

- @Component is a class-level annotation
 - @Component is used on our classes (i.e., classes for which we have the code), so Spring knows that it should add it. However, it only works, if we enabled a component scan (@ComponentScan) for our application and our class is included
 - @Component is a generic stereotype for any Spring-managed component. @Repository, @Service, and @Controller are specializations of @Component for more specific use cases (in the persistence, service, and presentation layers, respectively)
- @Bean is a method-level annotation
 - It is used on methods within classes usually annotated with @Configuration but also @Component
 - @Bean is used to gather beans at runtime, even to turn third-party classes into Spring beans
 - By annotating methods with @Bean, Spring can store the method's return as a Spring bean so that both a local class and an external (third-party) class can be turned into a Spring bean
 - By default, it will use the name of the method as the bean id/name, but you can specify a different name in the @Bean annotation

Summing up

- Spring is a popular Java framework that provides a number of features to simplify the development of enterprise applications
- One of the key features of Spring is the **Inversion of Control (IoC)** container, which **manages objects** and **their dependencies**, through **Dependency Injection**
- In Spring, you can define **beans**, which are **objects that are managed by the IoC container**
- Two of the most commonly used annotations for defining beans are **@Bean** and **@Component**

Summing up

@Bean is used to define bean methods within a configuration class

- @Bean can also be used to define a bean for third-party or non-Spring classes, such as a third-party library. This allows you to create a bean instance and customize its behavior as needed.
- @Bean methods can have arguments that can be used to inject dependencies into the bean instance. This is useful when you need to customize the behavior of a bean based on its dependencies.

@Component is used to mark a class as a component and is typically used for classes that you create yourself and that have a simple initialization process

- This is because @Component provides a convenient way to register a class as a bean without having to define a separate configuration class.
- @Component classes typically use constructor or setter injection to inject dependencies. This is because @Component classes do not have a bean method that can be used to inject dependencies.

Spring Boot

Spring Boot offers a fast way to build applications

- It looks at your classpath and at the beans you have configured, makes reasonable assumptions about what you are missing, and adds those items

With Spring Boot, you can focus more on
business features and less on infrastructure

For examples:

- **Is Spring MVC on the classpath?** There are several specific beans you almost always need, and Spring Boot adds them automatically. A Spring MVC application also needs a servlet container, so Spring Boot automatically configures embedded Tomcat.
- **Is Jetty on the classpath?** If so, you probably do NOT want Tomcat but instead want embedded Jetty. Spring Boot handles that for you.
- **Is Thymeleaf on the classpath?** If so, there are a few beans that must always be added to your application context. Spring Boot adds them for you.

<https://spring.io/guides/gs/spring-boot/>

Spring Initializr

- Spring Initializr allows you to generate JVM-based projects quickly,
 - Maven projects or Gradle projects
- You can customize the project to generate:
 - the build system and packaging
 - the language (Java, Kotlin, Groovy)
 - the packaging (jar, war)
 - the Spring Boot version
 - the dependencies to add to the project (e.g., Spring Web, Spring WS, Spring MVC)

<https://start.spring.io/>

Bear in mind ...

From previous slides

- When creating web services, there are two main development styles:
contract-last and contract-first

Contract-last approach

- Start with the Java code and let the web service contract (i.e., the WSDL) be generated from that

Contract-first approach

- Start with a WSDL contract and generate Java objects to implement the service

Spring-WS supports only the contract-first (and data-first) development style

Spring-WS (“marketing” slide 😊)

- Spring WS aims to facilitate **contract-first SOAP service development**, and this means that you need to start from a contract definition (e.g., XSD or WSDL)
- For that purpose, Spring WS allows for the creation of flexible web services using **one of the many ways to manipulate XML payloads**
- The product is based on Spring itself, which means **you can use the Spring concepts**, such as **dependency injection**, as an integral part of your Web service
- People use Spring-WS for many reasons, but most are drawn to it after finding alternative SOAP stacks **lacking when it comes to following Web service best practices**

Spring-WS makes the best practice an easy practice,
among which Contract-First development

Spring-WS (technical “marketing” slide 😊)

The other key features of Spring Web Services are:

- **Powerful mappings**
 - e.g., distributing incoming XML requests to any object, depending on message payload
- **XML API support**
 - e.g., JAXP APIs, such as DOM, SAX, and StAX, but also with JDOM, dom4j, XOM
- **Flexible XML Marshalling**
 - e.g., **JAXB1** and **JAXB2**
- **Reusing Your Spring expertise**
 - the architecture of Spring-WS resembles that of Spring-MVC
- **Support for WS-Security**
 - e.g., sign SOAP messages, encrypt and decrypt them, or authenticate against them
- **Integration with Spring Security**
 - the WS-Security implementation of Spring Web Services provides integration with Spring Security

Spring-WS modules

- The **XML module** (`spring-xml.jar`) contains various XML support classes for Spring WSs
 - It can be said that this is an internal module since it is mainly intended for the Spring-WS framework itself and not web service developers
- The **Core module** (`spring-ws-core.jar`) is the central part of the Spring's web services functionality
 - This module is mainly intended for the Spring-WS framework itself and not web service developers
- The **Support module** (`spring-ws-support.jar`) contains additional transports, not only HTTP
 - JMS, Email, and others
- The **Security package** (`spring-ws-security.jar`) provides a WS-Security implementation that integrates with the core web service package
 - You can sign, decrypt and encrypt, and add principal tokens to SOAP messages. Additionally, it lets you use your existing Spring Security security implementation for authentication and authorization

Terminology

A **Meta Annotation** is an annotation that is declared on another annotation. An annotation is, therefore, meta-annotated if it is annotated with another annotation.

A **Stereotype Annotation** is an annotation used to declare the **role** that a **component** plays within the application. For example, the `@Repository` annotation in the Spring Framework is a marker for any class that fulfills the role or stereotype of a repository (also known as Data Access Object or DAO).

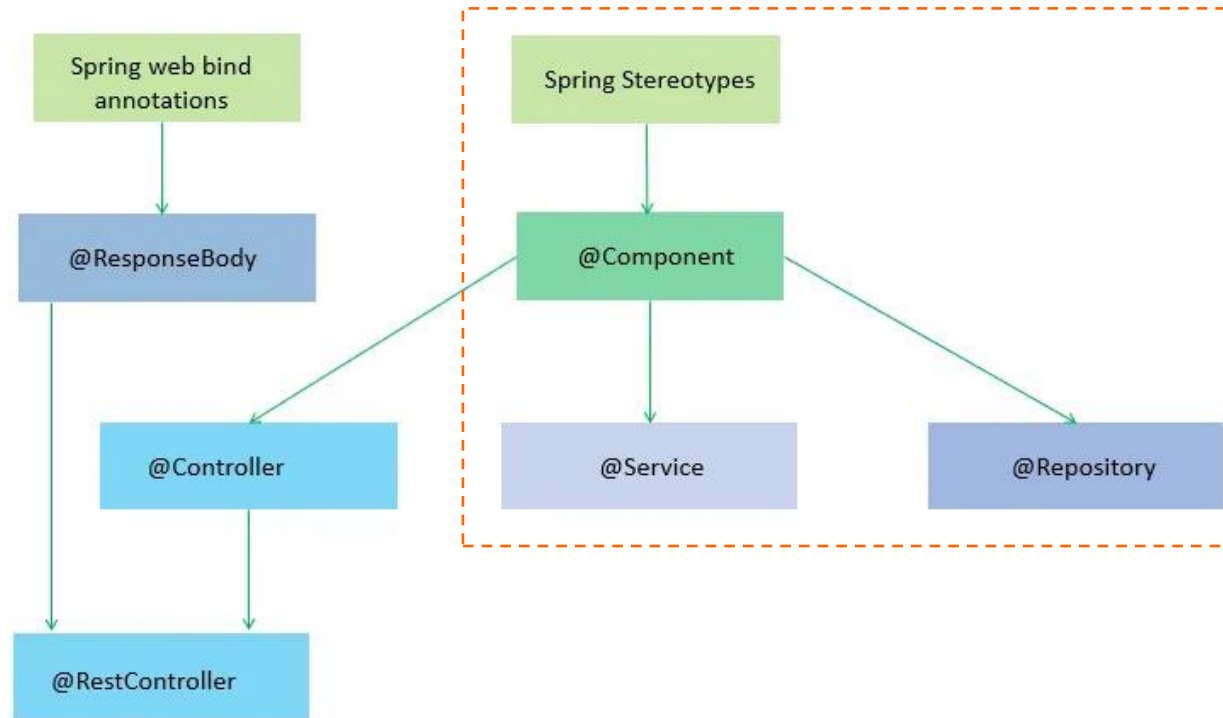
- `@Component` is a generic stereotype for any Spring-managed component. Any component annotated with `@Component` is a candidate for component scanning. Similarly, any component annotated with an annotation that is itself meta-annotated with `@Component` is also a candidate for component scanning. For example, `@Service` is meta-annotated with `@Component`.
- Core Spring provides several stereotype annotations out of the box, including but not limited to: `@Component`, `@Service`, `@Repository`, `@Controller`, `@RestController`, and `@Configuration`. `@Repository`, `@Service`, etc. are specializations of `@Component`.

Core Spring Annotations

- **@Bean** - The **annotated method** produces a bean managed by the Spring IoC container

Stereotype annotations

- **@Component** - Marks **annotated class** as a bean found by the component scanning and loaded into the application context
- **@Configuration** - Marks annotated class as a Java configuration defining beans
- **@Service** - Marks annotated class as a bean (as convention usually containing business logic)
- **@Repository** - Marks annotated class as a bean (as convention usually providing data access) and adds auto-translation from `SQLException` to `DataAccessExceptions`

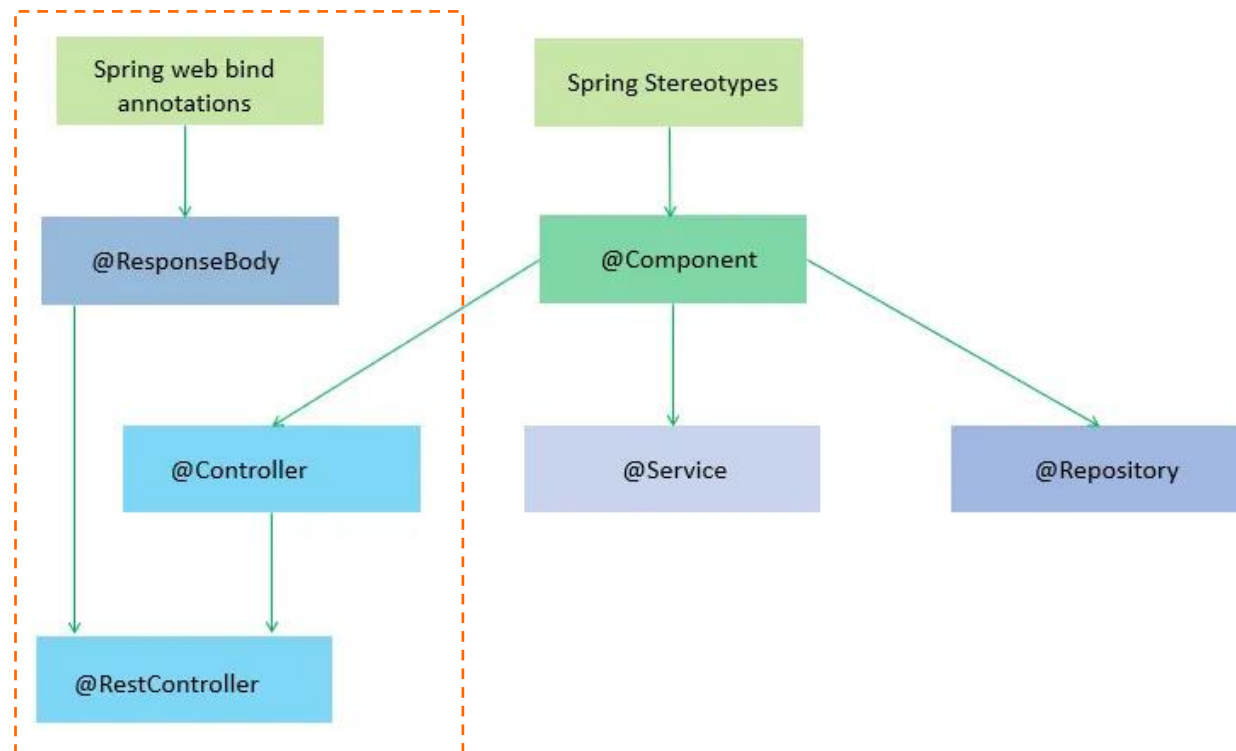


Core Spring Annotations

- `@Bean` - The **annotated method** produces a bean managed by the Spring IoC container

Stereotype annotations

- `@Component` - Marks **annotated class** as a bean found by the component scanning and loaded into the application context
- `@Configuration` - Marks annotated class as a Java configuration defining beans
- `@Controller` - Marks annotated class as a bean for Spring MVC containing request handler
- `@RestController` - Marks annotated class as a `@Controller` bean and adds `@ResponseBody` to serialize returned results as messages



Dependency Injection made easy

Types of Dependency Injection

Constructor Injection

- This is the most recommended way to achieve DI in Spring

Setter Injection

- Setter injection allows the dependency to be injected through a setter method

Field Injection

- It involves annotating the dependency field directly with `@Autowired`

Initialize our Dependency Injection project



Project

- ☐ Gradle - Groovy
☐ Gradle - Kotlin
☒ Maven

Language

- ☒ Java ☐ Kotlin
☐ Groovy

Spring Boot

- ☐ 3.5.0 (SNAPSHOT) ☐ 3.5.0 (M3)
☐ 3.4.5 (SNAPSHOT) ☒ 3.4.4
☐ 3.3.11 (SNAPSHOT) ☐ 3.3.10

Project Metadata

Group

Artifact

Name

Description

Package name

Packaging ☒ Jar ☐ War

Java ☐ 24 ☒ 21 ☐ 17

Dependencies

ADD ... ⌘ + B

Spring Web WEB

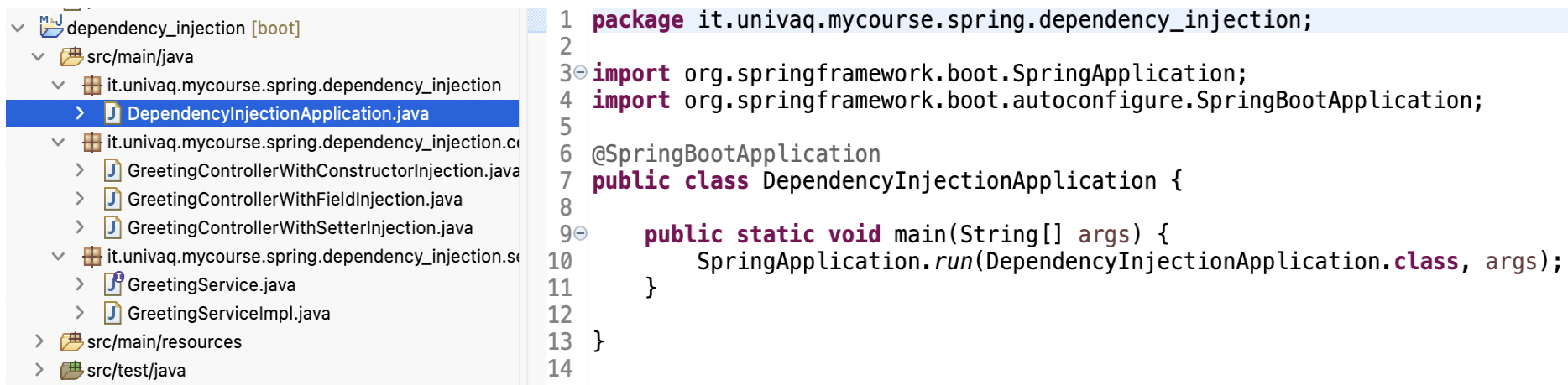
Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

GENERATE ⌘ + ↵

EXPLORE CTRL + SPACE

...

Spring Boot Application



```
1 package it.univaq.mycourse.spring.dependency_injection;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 @SpringBootApplication
7 public class DependencyInjectionApplication {
8
9     public static void main(String[] args) {
10         SpringApplication.run(DependencyInjectionApplication.class, args);
11     }
12
13 }
14
```

This class has been generated by Spring Boot so to make our Spring WS example as an application executable, which can be "just run"

The `@SpringBootApplication` is a convenience annotation that adds all of the following:

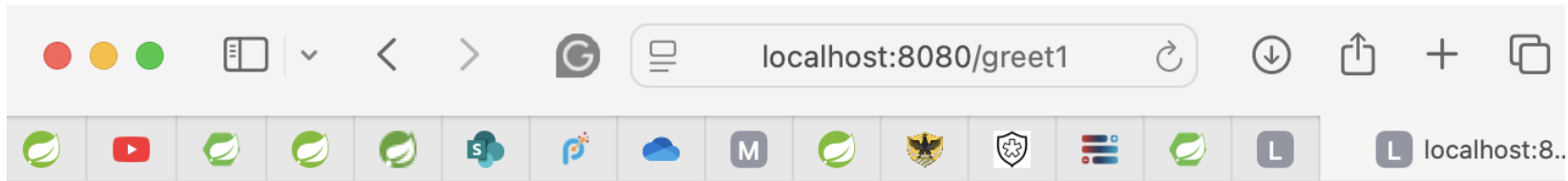
@Configuration: Tags the class as a source of bean definitions for the application context.

@EnableAutoConfiguration: Tells Spring Boot to start adding beans based on classpath settings, other beans, and various property settings. For example, if `spring-webmvc` is on the classpath, this annotation flags the application as a web application and activates key behaviors, such as setting up a `DispatcherServlet`.

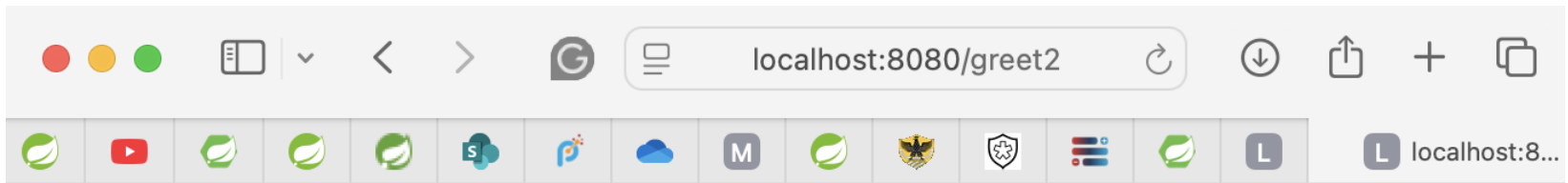
@ComponentScan: Tells Spring to look for other components, configurations, and services **in the package where the application is located**, letting it find the controllers.

-
- Screenshots to be added

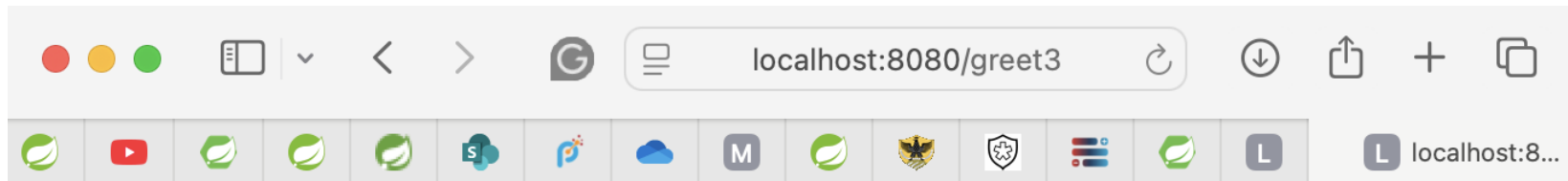
Test our “dependencies”



Hello, Spring Constructor Injection!



Hello, Spring Setter Injection!



Hello, Spring Field Injection!